Siga las instrucciones para realizar la configuración y creación de pruebas unitarias en react

El presente documento tiene como propósito guiar la incorporación de **pruebas automatizadas en proyectos React desarrollados con Vite**, utilizando el framework **Vitest** junto con **Testing Library**.

Esta configuración representa el estándar moderno para entornos frontend basados en React, al ofrecer un ecosistema **ligero, rápido y completamente integrado con Vite**, permitiendo evaluar componentes, hooks e interacciones del usuario de manera sencilla y eficiente.

1) Instalar dependencias

Ejecuta el siguiente comando para instalar todo lo necesario:

```
npm install --save-dev vitest @vitest/coverage-v8 @testing-library/react
@testing-library/jest-dom @testing-library/user-event jsdom
```

Paquete	Descripción
vitest	Framework de testing rápido e integrado con Vite
@vitest/coverage-v8	Genera reportes de cobertura (usa motor V8)
@testing-library/react	Pruebas de componentes React (render, queries, etc.)
@testing-library/jest-dom	Extiende los matchers (toBeInTheDocument, etc.)
@testing-library/user-event	Simula interacciones de usuario realistas (clics, tipeos, etc.)
jsdom	Crea un DOM virtual para ejecutar las pruebas en Node

2) Configurar Vitest en vite.config.js

Agrega esta configuración:

```
import { defineConfig } from "vite";
import react from "@vitejs/plugin-react";

export default defineConfig({
  plugins: [react()],
  base: "./",
  test: {
    globals: true,
    environment: "jsdom",
    setupFiles: "./tests/setup.js",
    coverage: {
      provider: "v8",
      reporter: ["text", "html"],
    },
  },
});
```

3) Configuración global de pruebas

Crea una carpeta en la raíz del proyecto llamada tests, en dicha carpeta crea el archivo setup.js:

tests/setup.js

```
import "@testing-library/jest-dom";
```

Con esta configuración, ya podrás ejecutar tus test unitarios, a continuación crearemos un componente y sus pruebas asociadas, para ver como realizar un enfoque de testing unitarios en react.

4) Crear un componente de ejemplo

Crea src/pages/Home.jsx:

```
import React, { useState } from "react";

function Home() {
  const [mensaje, setMensaje] = useState("");

  const validar = () => {
    setMensaje("Validado correctamente");
  };

  return (
    <>
      <main>
        <h1>Titulo del sitio</h1>
        <p>Lorem ipsum dolor sit amet consectetur adipisicing elit.
Reprehenderit iure fugiat mollitia saepe cupiditate nisi, deserunt
soluta.</p>
      </main>
      <button onClick={validar}>Validar</button>
      {mensaje && <p role="status">{mensaje}</p>}
    </>
  );
}

export default Home;
```

5) Crear pruebas del componente

Crea en la carpeta tests un archivo spec, donde configuraremos las pruebas unitarias:

tests/Home.spec.jsx

```
import React from "react";
import { render, screen, fireEvent } from "@testing-library/react";
import Home from "../src/pages/Home";

describe("Componente Home", () => {
  it("renderiza el título correctamente", () => {
    render(<Home />);
    expect(screen.getByText("Titulo del sitio")).toBeInTheDocument();
  });

  it("contiene un párrafo descriptivo", () => {
    render(<Home />);
    expect(screen.getByText(/Lorem ipsum/i)).toBeInTheDocument();
  });

  it('renderiza el botón "Validar"', () => {
    render(<Home />);
    expect(screen.getByRole("button", { name: /validar/i
})).toBeInTheDocument();
  });

  it("muestra mensaje al hacer clic en Validar", () => {
    render(<Home />);
    const boton = screen.getByRole("button", { name: /validar/i });
    fireEvent.click(boton);
    expect(screen.getByRole("status")).toHaveTextContent("Validado
correctamente");
  });
});
```

6) Scripts útiles en package.json

Agrega o actualiza los scripts, donde añadiremos los scripts **test, test:ui y test:coverage,** los cuales nos ayudarán a ejecutar las pruebas unitarias en el proyecto

```
{
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "preview": "vite preview",
    "test": "vitest",
    "test:ui": "vitest --ui",
    "test:coverage": "vitest run --coverage"
  }
}
```

7) Ejecutar pruebas

Modo consola:

```
npm run test
```

Modo visual (UI):

```
npm run test:ui
```

Nota: Para la ejecución del modo ui, la primera vez solicitará la incorporación de una dependencia, a lo cual debemos responder que si la instale (Y)

Reporte de cobertura:

```
npm run test:coverage
```

Nota: Para la ejecución del modo coverage, la primera vez solicitará la incorporación de una dependencia, a lo cual debemos responder que si la instale (Y)

Abre el reporte en coverage/index.html.

```
open coverage/index.html
```

8) Buenas prácticas recomendadas

- Prueba lo que el usuario ve o hace, no la implementación interna.
- Usa user-event para simular clics o escritura de forma realista.
- Mantén los tests simples y descriptivos.
- Organiza tus pruebas en una carpeta tests/ al mismo nivel que src/.